



VRIJE UNIVERSITEIT AMSTERDAM

Project Formalization in Lean: The AKS primality test

Project Computer Assisted Proofs

– FINAL REPORT –

Group members:

LUCIANO BALDASSI
JUSTUS DE BRUIJN KOPS
MARK LAPIDUS
DANIELA MACKAY
MARTIN RODRIGUEZ
TRISTAN VERMEULEN

Instructors:

JAN BOUWE VAN DEN BERG
SANDER DAHMEN

TAs:

LINDSEY VAN DER AALST
ANNE BAANEN
ALAIN CHAVARRI VILLARELLO

Contents

1	Introduction	2
2	The AKS Leap	3
3	Theorem of Agrawal, Kayal, and Saxena	4
3.1	Proof overview	4
4	Implementation of the AKS algorithm in Sage	5
5	Lean 4	6
5.1	Our approach	6
5.2	Example Formalization	7
5.3	Challenges	8
6	Conclusion	9
A	Link to code	10

Abstract

The primary focus of this project is the implementation of the AKS (Agrawal-Kayal-Saxena) primality test in Lean, a powerful proof assistant known for its formal verification capabilities. We will first give an introduction as to why we care about this specific primality test, as well as providing a compact overview of the correctness proof, which helps to establish a strategy for the Lean implementation. To check the algorithm itself, we used the environment SageMath. This choice of software can be explained by its strong support for symbolic computations, rather than numerical approximations. Finally, after getting familiar with the AKS algorithm and proof, we try to formalize the latter in Lean 4. The complete code can be found on Github in appendix A. The reader might notice that the formalization of the whole theorem is not finished. Towards the end of our paper we show the challenges and difficulties we encountered along the way, as well as some examples of Lean code.

1 Introduction

The problem of primality testing - determining whether a given number is prime - has been a central topic in number theory for over two millennia. Early methods for primality testing were relatively inefficient. One simple approach, trial division, involved checking the divisibility of the given number by all smaller primes. However, this method becomes impractical for larger numbers, since it requires to check $\sqrt{n}$ divisors of n in the worst case - and $\sqrt{n}$ grows exponentially in the number of digits of n .

More sophisticated primality tests were developed along the years, but were either computationally inefficient¹, non-deterministic², or relied on the unproven generalised Riemann hypothesis³.

The idea of the method we explore in this report is based on the well-known Fermat's Little Theorem:

Theorem 1 (Fermat's Little Theorem). *If n is prime, then n divides $a^n - a$ for all integers a .*

The contrapositive of which implies that if n does not divide $a^n - a$ for some integer a , then n cannot be prime.

The trouble is, the converse of this statement is not true. That is, if n is composite there is some a for which n does divide $a^n - a$.

Example

Consider a composite number $341 = 11 \times 31$ and take $a = 2$. Then $2^{341} - 2$ is divisible by 341.

But notice that for $a = 3$ above we get $3^{341} \equiv 168 \pmod{341}$ which shows that 341 is composite. So we might ask if we can always find such number a that would tell us if a composite n is actually composite.

It turns out, we cannot. There are certain numbers, called Carmichael numbers, that pass the Fermat test for every a coprime to n . In fact, there are infinitely many of them [1].

Example

Consider a composite number $561 = 3 \times 11 \times 17$, which divides $a^{561} - a$ for all integers a .

And so we see that the Fermat test does not always work. Nevertheless, it was a cornerstone for the AKS primality test by Agrawal, Kayal, and Saxena which made a major breakthrough in the history of primality testing. The AKS test is the first to be simultaneously general, polynomial-time, deterministic, and unconditionally correct. Unfortunately, despite its theoretical importance, the AKS test is rarely used in practice, since it outmatches other algorithms only for the very large numbers.

¹ See elliptic curve primality testing at https://en.wikipedia.org/wiki/Elliptic_curve_primality.

² See Baillie-PSW test at https://en.wikipedia.org/wiki/Baillie%E2%80%93PSW_primality_test.

³ See Miller-Rabin test at .

2 The AKS Leap

The idea behind the AKS test is based on the corollary of the [Fermat's Little Theorem](#); which since it's similar to the Child's binomial Theorem (also known as Freshman's dream) we will call it the modified Child's binomial Theorem.

Theorem 2 (Child's Binomial Theorem). *Given an integer $n \geq 2$, for all integers a coprime to n , n is prime if and only if*

$$(X + a)^n \equiv X^n + a \pmod{n} \quad (1)$$

in $\mathbb{Z}[X]$.

Proof. We begin by rewriting $(X + a)^n$ using the Binomial theorem:

$$\begin{aligned} (X + a)^n &= \sum_{k=0}^n \binom{n}{k} X^{n-k} a^k \\ &= \sum_{k=0}^n \frac{n!}{(n-k)! k!} X^{n-k} a^k \\ &= \frac{n!}{(n-0)! 0!} X^n + \sum_{k=1}^{n-1} \frac{n!}{(n-k)! k!} X^{n-k} a^k + \frac{n!}{(n-n)! n!} a^n \\ &= X^n + \sum_{k=1}^{n-1} \binom{n}{k} X^{n-k} a^k + a^n. \end{aligned}$$

To proceed with the proof, we utilise the following result :

Lemma 1. *n is prime if-and-only-if $\binom{n}{k} \equiv 0 \pmod{n}$ for $k \in \{2, 3, 4, \dots, n-1\}$.*

Continuing, we consider two directions:

($\implies$): Assume that n is prime. Then $a^n \equiv a \pmod{n}$ by the [Fermat's little theorem](#). By Lemma 1, we have $\sum_{k=1}^{n-1} \binom{n}{k} X^{n-k} a^k \equiv 0 \pmod{n}$. And so it follows that $(X + a)^n \equiv X^n + a \pmod{n}$.

($\impliedby$): We consider a contra-positive of the original statement where n is now composite. Let $p \neq a$ be a prime dividing n . Since a is coprime to n , p does not divide a . This means we can only consider the coefficients of the binomial expansion:

$$\binom{n}{p} = \frac{n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-(p-1))}{p \cdot (p-1) \cdot \dots \cdot 1}.$$

We see that p only divides n in the numerator and p in the denominator. If we now consider p^k as the largest power p dividing n , then it won't divide the denominator. And so p^{k-1} would be the largest power of p dividing $\binom{n}{p}$ from which follows that n does not divide $\binom{n}{p}$.

□

Note that the theorem 2 is itself a valid primality test. We could compute $(X + a)^n - (X^n + a) \pmod{n}$ and check whether n divides each coefficient. But since computing $(X + a)^n \pmod{n}$ requires checking $n + 1$ terms in the binomial expansion, it becomes computationally expensive for large n . For that reason, it is more optimal to also reduce mod some small degree polynomial, so that neither the coefficients nor the degree get large.

Consider the smallest such polynomial $(X^r - 1)$. Dividing $(X + a)^n$ by $(X^r - 1)$ leaves us with the remainder of r terms as opposed to $n + 1$. The question, of course, is how to determine the optimal parameter r and if it even exists. Note that

- the bigger the r , the smaller the ideal becomes, and the more coefficients there are to check in the original polynomial,
- if r is too small, however, we might erroneously decide composites as primes as the ideal becomes bigger and there are more terms that could be grouped together.

This is what we are going to investigate next.

3 Theorem of Agrawal, Kayal, and Saxena

The AKS algorithm is based on the following theorem:

Theorem 3 (Agrawal, Kayal and Saxena). *Given an integer $n \geq 2$, let $0 \leq r < n$ for which $\text{ord}_r(n) > (\log n)^2 \pmod{r}$. Then n is prime if-and-only-if*

- n is not a perfect power,
- n does not have any prime factor $\leq r$,
- $(X + a)^n \equiv X^n + a \pmod{(n, X^r - 1)} \forall a \in \mathbb{Z}$ such that $1 \leq a \leq \sqrt{r} \log n$.

3.1 Proof overview

The goal of this section is to give an outline of the correctness proof of theorem 3. We will not focus on a rigorous step by step proof as we are mostly interested in implementing it in Lean, however we still need to be able to understand it to use the proof-assistant.

The forward direction of theorem 3 is analogous to the proof of the forward direction of theorem 2, because every polynomial congruent to zero $\pmod{p}$ is also congruent to zero $\pmod{p, X^r - 1}$.

The reverse direction requires some more elaborate mathematics. We will prove the implication by contradiction, assuming that n is composite and that all conditions of theorem 3 hold. Let p be a prime number dividing n and $h(x) \in (\mathbb{Z}/p\mathbb{Z})[x]$ an irreducible factor of $x^r - 1$. It then follows that:

$$(X + a)^n \equiv X^n + a \pmod{n, x^r - 1} \implies (X + a)^n \equiv X^n + a \pmod{p, h(x)} \quad (2)$$

Note that we are now dealing with the quotient ring $\mathbb{F} := (\mathbb{Z}[x]/(p, h(x)))$ where the ideal $(p, h(x))$ is irreducible. Hence, it is isomorphic to a field, from which we can take the cyclic multiplicative group $\mathbb{F} \setminus \{0\}$, this will be important for the construction of some groups ahead. The reduction in (2) has the purpose of allowing us to deal with a field, where the congruence are easier to work with.

To continue with the prove we have to define a few sets first:

- Let H be the set of elements $\pmod{p, x^r - 1}$ generated multiplicatively by $x, x + 1, \dots, x + [A]$.
- Let G be a (cyclic) subgroup of $\mathbb{F}$ generated in the same way as H , so it's a reduction of $H \pmod{p, h(x)}$.
- Let S be set of positive integers k for which $g(x^k) \equiv g(x)^k \pmod{p, x^r - 1}$ for all $g(x) := \prod_{0 \leq a \leq A} (x + a)^{e_a} \in H$ where e_a is an arbitrary exponent dependent on a .
- Let R be the subgroup of $(\mathbb{Z}/r\mathbb{Z})^*$ generated by n and p .

We now will find contradicting bounds on $|G|$.

Upper bound

Consider the integers of the form $n^i p^j$ with $0 \leq i, j \leq \sqrt{|R|}$. Since n is not a power of p , they are distinct. It also holds that there are more integers $n^i p^j$ than $|R|$ so two must be congruent, namely, $n^i p^j \equiv n^I p^J \pmod{r}$ for some other combination $0 \leq I, J \leq \sqrt{|R|}$. We now use two lemmas from [2]. The first one states that for all $a, b \in S$, we have that $ab \in S$. The second says that if $a, b \in S$ and $a \equiv b \pmod{r}$, then $a \equiv b \pmod{|G|}$. It then follows that:

$$|G| \leq |n^i p^j - n^I p^J| \leq (np)^{\sqrt{|R|}} - 1 \leq n^{2\sqrt{|R|}} - 1$$

According to Andrew Granville, n/p is also in S , and can substitute n in the process above, to give a smaller upper bound: $|G| \leq n^{\sqrt{|R|}} - 1$.

Lower bound

To give a lower bound on $|G|$ we must show that there are many distinct element in G .

Remember that R was generated multiplicatively by n and p in $(\mathbb{Z}/r\mathbb{Z})$, hence it contains all elements of the form $n \pmod{r}$. By assumption, the order of $n \pmod{r}$ is greater than $(\log n)^2$, so R has at least more than $(\log n)^2$ elements. We can now say that $|R| > B$ where $B = \lfloor \sqrt{|R|} \log n \rfloor$.

Let T be a proper subset of $\{0, 1, 2, \dots, B\}$. So, $|T| \leq B < |R|$. If we consider the polynomials

$$\prod_{a \in T} (x + a) \tag{3}$$

we realize that they are elements of G and their degree is the number of elements $a \in T$, namely, $|T|$ which is less than $|R|$, this will be important to invoke the lemma from [2].

Lemma 2. Suppose that $f(x), g(x) \in \mathbb{Z}[x]$ with $f(x) \equiv g(x) \pmod{p, h(x)}$ such that their reductions in $\mathbb{F}$ are in G . If $\deg(f(x)), \deg(g(x)) < |R|$, then $f(x) \equiv g(x) \pmod{p}$

By taking the contrapositive of the lemma above, it follows that the polynomials 3 are distinct in G if they are distinct in $(\mathbb{Z}/p\mathbb{Z})[x]$ for every distinct T . Hence we just need to count the possible sets T which is $2^{B+1} - 1$. So we can write: $|G| > 2^{B+1} - 1$. We now just have to show that $2^{B+1} - 1 > n^{\sqrt{|R|}} - 1$:

$$\begin{aligned} 2^{B+1} - 1 > n^{\sqrt{|R|}} - 1 &\iff 2^{B+1} > n^{\sqrt{|R|}} \iff \log(2^{B+1}) > \log(n^{\sqrt{|R|}}) \iff B + 1 > \sqrt{|R|} \log n \\ &\iff \lfloor \sqrt{|R|} \log n \rfloor + 1 > \sqrt{|R|} \log n \end{aligned}$$

Hence we proved that $|G| > n^{\sqrt{|R|}} - 1$, contradicting the upper bound of $|G|$ we got before.

4 Implementation of the AKS algorithm in Sage

We next consider the AKS algorithm, given the above proof we have actually verified that the following steps produce a prime number:

1. We find out if n can be written as $n = a^b$ for $a, b > 1$. If this can be done, we notice that $n = a(a^{b-1})$, hence n is a composite number.
2. Find the smallest r so that the order of $n \pmod{r} > (2 \log n)^2$. The reason why r has to be small is important later, and the fact that $n \pmod{r}$ is bounded is done to satisfy the lower bound on $|G|$.
3. Check for $a \in [2, \min r, n - 1]$ if $a|n$ if this happens output composite. This follows from a primality test for numbers.
4. If $n \leq r$ output prime.

5. For $a \in [1, \sqrt{\phi(t)2 \log n}]$ check if congruence $(X + a)^n = X^n + a \in \text{mod } X^r - 1, n$ is true if not output composite. This bound is to guarantee all elements that are generated are distinct.
6. Output prime.

Notice that this procedure is a sieving method. At each stage we sieve out non prime numbers until the last one where we apply the proven result. The reason we do this is because it might still be expensive to try and solve the last equation in one go. The code for this procedure is given below: To informally show that our method is correct we consider the first 10000 numbers and check them for primality. With we see this agrees with what is given in sage:

Last we verify using asymptotic law of distribution of prime numbers, for this we consider the primes up until $N = 104729$. We count the amount of primes the algorithm gives us, this is: $k = 10000$ dividing this by the amount of numbers we have counted gives us. To compare this with the theorem we compute $\frac{N}{\log N} \approx 9060$ this comes close to the actual value so we state the theorem is correct for this amount of numbers.

The SageMath script containing our implementation of the AKS algorithm is available of GitHub (cf. Appendix A).

5 Lean 4

We formalized our proof using *Lean 4*. Lean is an open-source, crowd-sourced, proof assistant based on dependant type theory and Calculus of Constructions. It is written in C++ developed and maintained by a diverse community of users, predominantly from mathematics and computer science. Proof assistants, also called interactive theorem provers, are software tools that help mathematician formalize proofs rigorously by expressing every theorem in term of axioms and checking that these proofs are correct down to their logical foundation. Proof assistant thus allow mathematicians to check their proof's correctness, but they also allow them to observe on which assumptions their proof and theorems do or don't rely on. To prove mathematical statements in Lean requires making large use of the *Mathlib* library. This library contains a substantial number of theorems, definitions, and proofs, as well as many tactics which makes theorem proving easier. It is also developed by the Lean community.

As often happens in mathematics, what started as a fun project on a niche topic has already started to show it's use by giving suggestions on what step to use next in a proof, and could perhaps be used to guide researchers in the future. The Lean project was launched in 2013 by Leonardo de Moura, and since 2018 it has grabbed the attention of leading mathematicians such as Terence Tao, Kevin Buzzard, Tomas Hals and Peter Scholze who have already been able to apply it to modern advances in mathematics. Such interest goes hand in hand with expanding the Lean library and has the potential to catalyze the development of new mathematics.

5.1 Our approach

The aim of this project was to implement in Lean as most as we could of the proof of correctness of the AKS primality test. It became immediately apparent from reading the first paragraph of Chapter 4 of Grandville's paper [2] that this was going to be a challenge. In such a short paragraph, six assumptions are already given, one of which is a congruence equation modulo $(n, x^r - 1)$. As a reader, one just skims these assumptions, knowing almost immediately what they mean and can proceed to go through the proof.

This is not the case with Lean. In order to formalize a proof, one has to state all assumptions in the shape of hypothesis from the beginning, and eventually write definitions to describe properties that are not implemented in Mathlib. Furthermore, the type of each variable must be declared precisely and condition strictly which operations can be done with them. This makes the choice of variable type essential as a two equivalent problems using different formulations might not be equally convenient to manipulate. Finally, facts that seem trivial or obvious and are usually skimmed over must be stated explicitly (such as something being non null), highlighting the importance of all of these assumptions.

We settled ourselves to formalize both [Child's Binomial Theorem](#) and the [Agrawal, Kayal and Saxena Theorem](#).

Eventually, we managed to prove the forward direction of both theorem (with, for the former, a significant assistance from our TA Anne Baanen), and part of the reverse direction. Our formalizations are available on the GitHub of the project (cf. Appendix A).

5.2 Example Formalization

To demonstrate writing a formal proof in Lean, we showcase the proof of the forward direction of the AKS algorithm here. For simplicity, we have already assumed that n is not a prime power nor has any prime factor smaller or equal to r . The first step of any proof is to declare the goal of the proof (the theorem that needs to be proven) as well as any hypothesis and assumptions this theorem relies on. Translating Theorem 3 into a Lean goal, we defined the following problem⁴:

```
Nat.Prime n ↔ ∀ a : ℕ, a ≤ (Real.sqrt ↑r) * Real.logb 2 ↑n → (X + C (a : ZMod n))n - (Xn + C (a : ZMod n)) ∈ Ideal.span {Xr - 1, C ↑n}
```

Notice that instead of attempting to prove $(x + a)^n \equiv x^n + a \pmod{(n, x^r - 1)}$, we reformulated the problem in the equivalent problem of showing that $(x + a)^n - (x^n + a)$ is an element of the ideal $(n, x^r - 1)$. This design choice allowed us to work almost exclusively in the same polynomial ring rather than having to deal with casting different types and handling the moduli repeatedly.

Similarly as for usual proof, the first step of proving an "iff" statement in Lean consist in splitting the proof into a subproof for the forward ($\Rightarrow$) direction and one for the reverse ($\Leftarrow$) direction. This is done with the `constructor` tactic. Lean then create two separate goal for each direction.

Our proof for the forward direction follows like:

```
1 case mp => -- Nat.Prime n → ∀ (a : ℕ), ↑a ≤ Real.sqrt ↑r * Real.logb 2 ↑n → (X + C ↑a)n - (Xn + C ↑a) ∈ Ideal.span {Xr - 1, C ↑n}
2   rintro hn a ha
3   have h₁ : (X + C (a : ZMod n))n - (Xn + C (a : ZMod n)) ∈ Ideal.span {C ↑n} := by
4     have : Fact (Nat.Prime n) := Fact.mk hn
5     rw [(childBinomial _ _ nGT1).mp hn, sub_self];
6     exact Ideal.zero_mem _
7   rw [Ideal.span_insert]
8   apply Ideal.mem_sup_right
9   exact h₁
```

This proof develops the argument presented in Sec. 3. Going over this code, and after selecting our forward goal to prove, we introduce the assumptions specific to this goal with `rintro`. This simplifies the goal by creating three new hypotheses for n being a prime number, $a \in \mathbb{N}$, and $a \leq \sqrt{r} \log_2(n)$, and naming them `hn`, `a`, and `ha`, respectively.

On line 3, the `have` tactic creates a new hypothesis `h₁` stating that $(x + a)^n - (x^n + a)$ is in the ideal generated by n , which we'll be able to use later. Line 4 to 6 then provide the proof of this new statement.

- Line 4 transforms the assumption `hn` and tells Lean to explicitly treat n as a prime number in all subsequent steps.
- Line 5 then uses the `rw` tactic to rewrite $(X + a)^n - (X^n + a)$ into zero using the theorems written inside the brackets. The first rewrite uses `childBinomial` to rewrite $(X + a)^n$ into $(X^n + a)$. For `childBinomial` to be used, we need to provide Lean with the hypotheses necessary to verify that the theorem hold. Here, the only hypothesis we have to explicitly write is `nGT1` (stating that $n > 1$) which is an assumption declared with our problem; the other hypotheses are inferred by Lean as indicated by the underscores. The suffix `.mp` after the parentheses indicates that we want to use the forward implication of `childBinomial`. Rest to satisfy the theorem's premise that n is prime, which is given by the hypothesis `hn`. The second rewrite expresses $(X^n + a) - (X^n + a)$ into zero and is done by invoking `sub_self`. This theorem simply states that the difference of two identical elements of an additive group is equal to zero. Note that unlike `sub_self`, the theorem `childBinomial` is not built-in in Mathlib but refer to a theorem that we previously proved and named thus.

- We are left to show that zero is an element of the ideal generated by n . This holds for any ideal; the `exact` tactic

⁴For concision sake, the declarations of the assumptions are omitted here. We refer the reader to the code available on GitHub for the full declaration of the problem.

on line 6 then matches our goal with theorem `Ideal.zero_mem`, which states exactly that, thus completing the proof of h_1 . Notice the use of the underscore to leave up to Lean to infer the appropriate ideal to use.

Finally, line 7 to 9 wrap up our original proof. Line 7 rewrites our goal (at this point, we are back with $(x + a)^n - x^n + a \in (n, x^r - 1)$) to express the ideal generated by two elements as the union of two ideals generated by one element. Line 8 the uses the `apply` tactic to invoke the theorem `Ideal.mem_sup_right`. This theorem states that if an element x is an element of an ideal T , then x is an element of the ideal S united by the right by T . We are left to show that our expression is in the ideal generated by n which is exactly the hypothesis we proved on line 3 to 6. Using the `exact` tactic one last time thus completes the proof.

5.3 Challenges

Working with Lean came with its lot of challenges, the first of which stemmed directly from the short time frame of the project. Indeed, the project being contained over three weeks from group formation to submission, this left us a single week to get acquainted with Lean and another one to attempt to formalize the proofs. This obviously capped our abilities to use the software and limited our understanding of the available tactics and constructions. We were hopefully assisted by our TAs whom unlocked us from being stuck several times.

Another issue we encountered, which we conjecture is linked to Lean still being in active development and the community of user being relatively small, is the scarcity of available tutorials and example for beginners. Aside from the 'Mathematics in Lean' tutorial available on the official website and the Zulip forum – which posts were often too specific and beyond our level – we found little documentation to clarify multiple of the rather basic questions we had.

We also found the documentation to be sometimes difficult to search for neophyte such as us. Even once acquainted with Lean naming conventions and hierarchy, finding the proper method often wasn't trivial. Most theorem also do not have natural language description, restraining our search terms to certain keywords and Lean's naming convention, leading to a lot of guess work. Although we expect this to be less of an issue for more seasoned Lean user, it definitely affected us.

Finding how to tell Lean what we wanted it to do was often quite hazardous. Classical (pen and paper) proof writing allow to go from one argument to the other and to manipulate constructions in ways that are not trivial to do in Lean. Finding how to properly use theorems and provide them with the appropriate hypotheses to manipulate our proof and work within the constraints imposed by the theorem prover caused us many a headache.

This difficulty of figuring out how to translate our ideas was especially tricky for defining certain subgoals. For instance, consider the definition of the set H introduced Sec. 3. The code below show the definition of that set alongside the definition of the ideal $I := (X^r - 1, n)$ and the quotient set $xSet := \mathbb{Z}_n[x]/I$ it relies on:

```
def I : Ideal (ZMod n)[X] :=
  Ideal.span {X^r - 1, C ↑n}

def xSet : Set ((ZMod n)[X] / (I r n)) := -- Set of x+1, x+2,...,x+A
  {P | ∃ (a : ℕ), 0 ≤ a ∧ a ≤ A ∧ P = Ideal.Quotient.mk (I r n) (X + C (a : ZMod n)) }

def HSet : (Set ((ZMod n)[X] / I r n)) := -- Set H
  {x | x ∈ Submonoid.closure (xSet A r n)}
```

The root of confusion here is the typing. We want here to create a subset of the quotient set created by the ideal I . By definition, ideals are sets embedded with a few extra properties, so we were expecting being able to use I as a set. But in Lean, our definition of I here is a function that given two inputs creates an element of type `Ideal`. Therefore, to define the type of `xSet` we have to include `Set` which casts the type `Ideal` into a set. The fact that I has type `Ideal` and not `Set` was rather counter intuitive, and finding the correct type to classify `xSet` under took a lot longer than expected. Similarly, in this example, we also have to cast the constant a so it is of type `Polynomial`.

6 Conclusion

This project gave us the opportunity to investigate the nature of primality tests and the underlying theory of the AKS test in specific. The process of formalization not only deepened our comprehension of the theory but encouraged us to consider multiple perspectives. By the end of the project we were acquainted with the basics of Lean and were able to put in practice our knowledge from other courses such as Rings and Fields.

References

- [1] C. P. W. R. Alford, Andrew Granville, “There are infinitely many carmichael numbers,” *Annals of Mathematics*, vol. 139, no. 3, pp. 703–722, 1994.
- [2] A. GRANVILLE, “It is easy to determine whether a given integer is prime,” *Bulletin of the American Mathematical Society*, vol. 42, September 2004.
- [3] M. Agrawal, N. Kayal, and N. Saxena, “Primes is in p,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.

Appendices

A Link to code

All SageMath and Lean code produced as part of this project are available on GitHub at https://github.com/Eejil/CAP-AKS_primality_test_project.